

Architektury systemów komputerowych

Lista zadań nr 8

Na zajęcia 22 kwietnia i 5 maja 2026

Należy być przygotowanym do wyjaśnienia semantyki każdej instrukcji, która pojawia się w treści zadania. W tym celu posłuż się dokumentacją: [x86 and amd64 instruction reference](http://www.felixcloutier.com/x86/)¹. W szczególności trzeba wiedzieć jak dana instrukcja korzysta z rejestru flag EFLAGS tam, gdzie obliczenia zależą od jego wartości. W trakcie tłumaczeniu kodu z assemblera x86-64 do języka C należy trzymać się następujących wytycznych:

- Używaj złożonych wyrażeń minimalizując liczbę zmiennych tymczasowych.
- Nazwy wprowadzonych zmiennych muszą opisywać ich zastosowanie, np. result zamiast rax.
- Instrukcja goto jest zabroniona. Należy używać instrukcji sterowania if, for, while i switch.
- Pętle while należy przetłumaczyć do pętli for, jeśli poprawia to czytelność kodu.

Zadanie 1. Przeczytaj poniższy kod w języku C i odpowiadający mu kod w assemblerze, a następnie wywnioskuj jakie są wartości stałych A i B.

```
1 typedef struct {
2     int x[A][B];
3     long y;
4 } str1;
5
6 typedef struct {
7     char array[B];
8     int t;
9     short s[A];
10    long u;
11 } str2;

12 void set_val(str1 *p, str2 *q) {
13     long v1 = q->t;
14     long v2 = q->u;
15     p->y = v1 + v2;
16 }

1 set_val:
2     movslq 8(%rsi),%rax
3     addq   32(%rsi),%rax
4     movq   %rax,184(%rdi)
5     ret
```

Zadanie 2. Przeczytaj poniższy kod w języku C i odpowiadający mu kod w assemblerze, a następnie wywnioskuj jakie są wartości stałych R, S i T.

```
1 long A[R][S][T];
2
3 long store_elem(long i, long j,
4                 long k, long *dest)
5 {
6     *dest = A[i][j][k];
7     return sizeof(A);
8 }

9 store_elem:
10    leaq   (%rsi,%rsi,2),%rax
11    leaq   (%rsi,%rax,4),%rax
12    movq   %rdi,%rsi
13    salq   $6,%rsi
14    addq   %rsi,%rdi
15    addq   %rax,%rdi
16    addq   %rdi,%rdx
17    movq   A(,%rdx,8),%rax
18    movq   %rax,(%rcx)
19    movq   $3640,%rax
20    ret
```

Zadanie 3. Przeczytaj poniższy kod w języku C i odpowiadający mu kod w assemblerze, a następnie wywnioskuj jaka jest wartość stałej CNT i jak wygląda definicja struktury a_struct.

```
1 typedef struct {
2     int first;
3     a_struct a[CNT];
4     int last;
5 } b_struct;
6
7 void test (long i, b_struct *bp) {
8     int n = bp->first + bp->last;
9     a_struct *ap = &bp->a[i];
10    ap->x[ap->idx] = n;
11 }

12 test:
13    movl   0x120(%rsi),%ecx
14    addl   (%rsi),%ecx
15    leaq   (%rdi,%rdi,4),%rax
16    leaq   (%rsi,%rax,8),%rax
17    movq   0x8(%rax),%rdx
18    movslq %ecx,%rcx
19    movq   %rcx,0x10(%rax,%rdx,8)
20    retq
```

¹<http://www.felixcloutier.com/x86/>

Zadanie 4. Posługując się ABI dla architektury x86-64 wyznacz rozmiar struktury `node`. Dla każdej składowej typu `node` podaj: wymaganie na wyrównanie (funkcja `alignof`), przesunięcie względem początku struktury `node` (funkcja `offsetof`), rozmiar (funkcja `sizeof`). Następnie zoptymalizuj rozmiar struktury zmieniając kolejność pól. Wyznacz rozmiar struktury po optymalizacji.

```

1 struct node {
2     char id[2];
3     int (*hashfn)(char *);
4     short flags;
5     union {
6         struct {
7             short n_key;
8             int n_data[2];
9             unsigned char n_type;
10        } s;
11        unsigned l_value[2];
12    } u;
13 };

```

Zadanie 5. Przeczytaj definicję unii `elem` i wyznacz jej rozmiar w bajtach. Następnie przepisz procedurę `proc` na kod w języku C.

```

1 union elem {
2     struct {
3         long *p;
4         long y;
5     } e1;
6     struct {
7         long x;
8         union elem *next;
9     } e2;
10 };

11 proc:
12     movq 8(%rdi),%rax
13     movq (%rax),%rdx
14     movq (%rdx),%rdx
15     subq 8(%rax),%rdx
16     movq %rdx, (%rdi)
17     ret

```

Zadanie 6. W jakim celu procesor używa predyktora skoków? Które skoki warunkowe warto zoptymalizować do instrukcji `cmov`? Posługując się narzędziem *Compiler Explorer* z odpowiednio dobranym poziomem optymalizacji (np. `-O0`, `-O1`, `-O2`, `-Os`) przepisz poniższą funkcję tak, żeby w wierszu 4 kompilator nie wygenerował skoku warunkowego (czyli w kodzie po optymalizacji powinny wystąpić tylko dwa skoki warunkowe).

```

1 void merge1(long src1[], long src2[], long dest[], long n) {
2     long i1 = 0, i2 = 0;
3     while (i1 < n && i2 < n)
4         *dest++ = src1[i1] < src2[i2] ? src1[i1++] : src2[i2++];
5 }

```

Zadanie 7 (bonus). Przeczytaj definicje struktur `SA` i `SB`, po czym przeanalizuj kod procedur o sygnaturach «`SB eval(SA s)`» i `long wrap(long x, long y, long z)`. Następnie zapisz w języku C kod odpowiadający procedurom `eval` i `wrap`. Narysuj diagram przedstawiający zawartość rekordu aktywacji procedury `wrap` w momencie wywołania funkcji `eval`.

```

1 typedef struct A {
2     long u[2];
3     long *v;
4 } SA;
5
6 typedef struct B {
7     long p[2];
8     long q;
9 } SB;

10 eval:
11     movq %rdi, %rax
12     movq 16(%rsp), %rcx
13     movq 24(%rsp), %rdx
14     movq (%rdx), %rsi
15     movq %rcx, %rdx
16     imulq %rsi, %rdx
17     movq %rdx, (%rdi)
18     movq 8(%rsp), %rdx
19     movq %rdx, %rdi
20     subq %rsi, %rdi
21     movq %rdi, 8(%rax)
22     subq %rcx, %rdx
23     movq %rdx, 16(%rax)
24     ret

25 wrap:
26     subq $72, %rsp
27     movq %rdx, (%rsp)
28     movq %rsp, %rdx
29     leaq 8(%rsp), %rax
30     pushq %rdx
31     pushq %rsi
32     pushq %rdi
33     movq %rax, %rdi
34     call eval
35     movq 40(%rsp), %rax
36     addq 32(%rsp), %rax
37     imulq 48(%rsp), %rax
38     addq $96, %rsp
39     ret

```